APERTUS MUNDUS

A persistent world, grounded in evidence

Technical report | HackApertus Track 2b | Team Apertus Mundus | 16 October 2026

1. Summary

Apertus Mundus is a persistent narrative simulation workbench for a game master (GM). It uses Apertus 70B to propose possible outcomes, while a Python engine owns entities, state, permissions and confirmed history. The central engineering question is how to sustain an evolving world without turning every interaction into a replay of the entire campaign.

Three connected mechanisms address this problem: short candidate summaries followed by expansion of only the selected branch; a source-linked Wiki and retrieval pipeline that assembles evidence for each request; and three layers of memory backed by deterministic state transitions. Generated possibilities remain drafts until a commit succeeds. Streaming provides incremental narrative feedback, while complete-response validation protects persistent state.

On a fixed Chinese campaign benchmark, the hosted reranking path increased Hit@6 from **25/30 to 30/30** and evidence Recall@6 from **26/33 to 31/33**. Median retrieval latency increased from **0.277 s to 2.905 s**. Separate Apertus traces show short options available in **15.55 s and 18.64 s**, with complete selected-branch workflows taking **40.32 s and 38.15 s**. These small-sample observations provide engineering evidence, rather than a general model-quality or throughput claim. [7-10]

2. Architecture: generation, validation and commitment

The browser workbench communicates with FastAPI. A context builder combines current structured state, scoped memory, original evidence and spatial constraints. Apertus returns candidate branches; the GM selects one, required entity or map ambiguities are resolved, and that branch is expanded over server-sent events (SSE). The application then submits the completed expansion through the transactional commit API. [12]

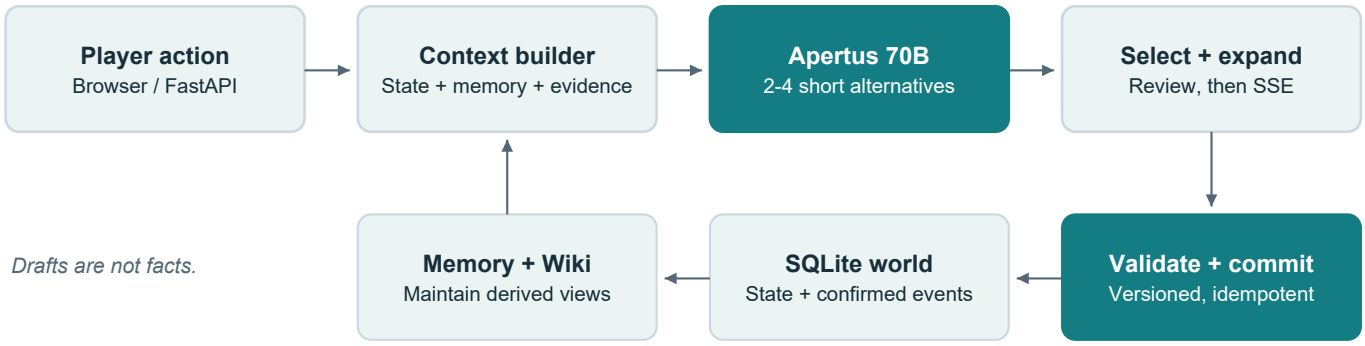


Figure 1. The world-update loop. Only the validated commit writes authoritative facts. Confirmed events subsequently feed memory and knowledge maintenance.

A candidate narrative, a relevance score and a Wiki page are distinct from a committed world fact. SQLite stores stable entity identities, typed state, events and team timelines. Operation keys make retries idempotent; state fingerprints and leases detect stale or concurrent work. A cancelled stream or rejected response cannot commit partial text. This separates open-ended language reasoning from the exact checks that maintain world consistency. [12]

The system evolves an existing narrative-workbench architecture. This report focuses on the current Apertus integration, staged generation, unified evidence pipeline, knowledge lifecycle and controlled commit path; it does not claim that the entire engine originated in one experiment.

3. Apertus inference: make the choice available first

Short options, then selected-branch expansion

Generating full prose for every possible outcome makes the user wait for text they may never read. The revised path asks Apertus for **2-4 distinct options with short summaries**, targets 40-80 Chinese characters per summary, and defers complete prose until selection. Empty effect fields are omitted. Unselected branches receive neither full expansion nor speculative entity/map review. Review decisions constrain the selected branch's first full expansion. [9,12]

SSE changes when useful text becomes visible; it does not remove generation work. Text stays provisional until the upstream response ends normally and branch/world versions still match. The prompt and optional JSON Schema request at most 100 characters; the server deliberately tolerates up to 120 characters, preserving modest overruns without truncation. Longer new summaries are rejected intact. Direct boundary regressions cover this policy; legacy full branches retain their existing contract. [9,12]

Reported improvement and measured timing

Team development observation [11]	Before	After	Interpretation
Apertus response, reported mean	60 s	18 s	70.0% less time; 3.33x faster
DeepSeek v4 Flash comparison	40 s	5 s	87.5% less time; 8.00x faster
Perceived waiting time	-	>80% lower	Informal experience claim

These retain the team's observations; paired traces, sample counts and common timing boundaries were not retained with the brief. In particular, 60 to 18 seconds is a **70%** reduction, not evidence of an over-80% measured latency reduction. DeepSeek is a team comparison configuration, not a dependency of the submitted system.

Retained Apertus v1.5-70B traces [8,9]	Earlier success	Revised B	Revised C
Options available	47.25 s	15.55 s	18.64 s
Expansion request to first displayable text	11.37 s	16.29 s	4.21 s
Complete expansion request	19.05 s	24.04 s	19.15 s
Full workflow, including commit	66.47 s	40.32 s	38.15 s

The revised run set contains two successes and one rejected overlong summary (19.13 s). Successful options average **17.09 s**; complete workflows average **39.23 s**. The harness uses one fixed action and an isolated database, excluding human reading, browser rendering/transport and background maintenance. Output and service timings differ, so this is not a controlled A/B study.

Why the change helps

Earlier valid responses contained 145-221 characters per branch against an approximately 60-character target. One non-streamed call emitted 1,886 completion tokens in 29.21 s, despite 8,320 of 8,329 prompt tokens being reported cached. Revised successful summaries emitted 380 and 467 completion tokens. Reducing unused output is therefore a concrete latency mechanism. [8,9]

Our working explanation is instruction dilution in a crowded context plus an over-demanding output contract. Narrower *effective attention to the task* is a hypothesis, not a measured architectural property. Apertus v1.5 advertises a 262,144-token context window; these traces do not show that nominal window size caused overruns or isolate differences from DeepSeek. Prompt-length and output-budget ablations are needed. [1,8]

4. Where do the prompts come from?

A maintained knowledge view, assembled for each action

Application-owned templates define role, output schema, draft status and permissible effects. The context builder adds world rules, active party state, registered state definitions, session history, L1 recent events, L2 entity records, retrieved evidence and map constraints. Routing can omit irrelevant source classes; current location and adjacency remain a spatial safety anchor. [12]

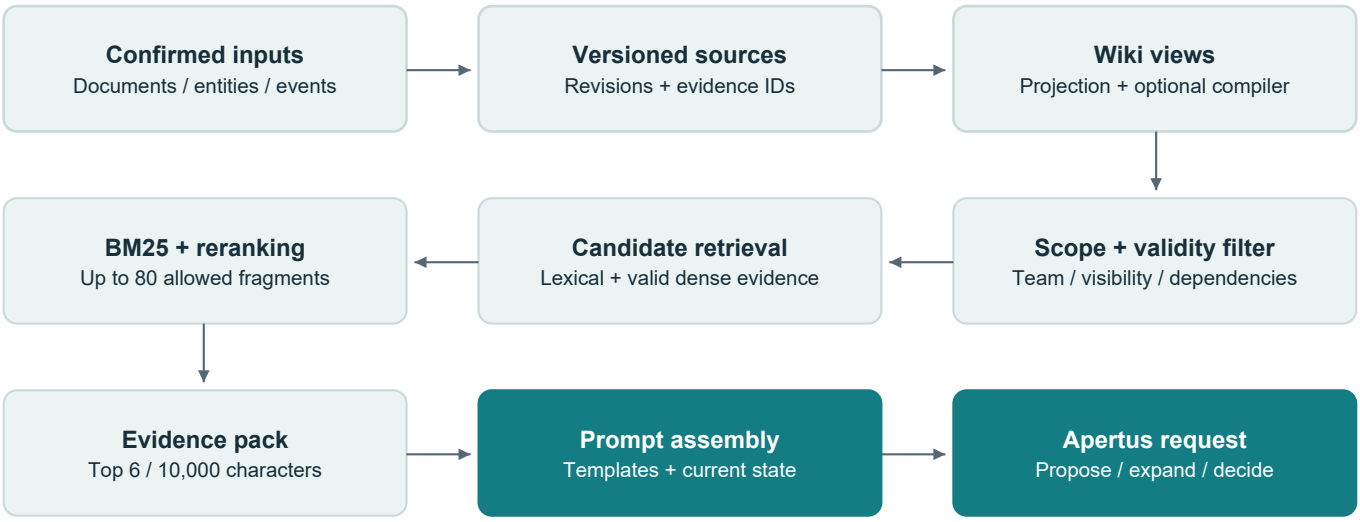


Figure 2. From source material to an actionable prompt. Deterministic projection is always available. Optional LLM compilation organizes original evidence; retrieval passes selected evidence and provenance into the inference prompt.

Wiki compilation preserves evidence and provenance

Imported documents, entity state and confirmed events become versioned sources. Deterministic projection creates evidence blocks and Wiki pages. The optional Apertus compiler selects evidence IDs into sections and topic pages instead of freely rewriting facts. Dependencies retain source revisions, and published text incorporates the selected original blocks. Proposed links and state claims remain review candidates; conflicts do not silently overwrite state. [12]

Compilation uses approximately 12,000 source characters per batch and at most 16 batches. Combined sources must match scope, team, kind and enabled status. After inference, publication rechecks source revisions and the world epoch. An edit, rollback or revocation invalidates stale work. Source removal invalidates dependent knowledge. Wiki maintenance writes knowledge views; it cannot execute a proposed world-state change. [12]

Filter first, retrieve broadly, rerank, then pack

The engine filters visibility, team scope, enabled status, current revision and valid dependencies before ranking. Lexical retrieval and valid dense embeddings operate within that set. With reranking enabled, block-level BM25, a weak title hint, exact-text deduplication and a source quota build at most **80 fragments**, with at most **12 per source**. Reranking precedes the final **6 evidence groups / 10,000-character** budget. Duplicate text retains all permitted provenance. [7,12]

Invalid or unavailable reranking falls back to initial ordering. A changed world fingerprint prevents stale evidence publication. Calls inside a write transaction skip external reranking, avoiding a write lock held through that request. One-hop expansion remains off by default: the large-campaign baseline reduced Hit@6 from 25/30 to 24/30 and evidence recall from 26/33 to 24/33 when it was enabled. [6,7]

This is the operational meaning of grounding: a bounded, attributable view of the world. Retrieval success measures access to evidence; it does not prove that the model understands or correctly uses it.

5. Containing context growth with memory and rules

Three layers of memory

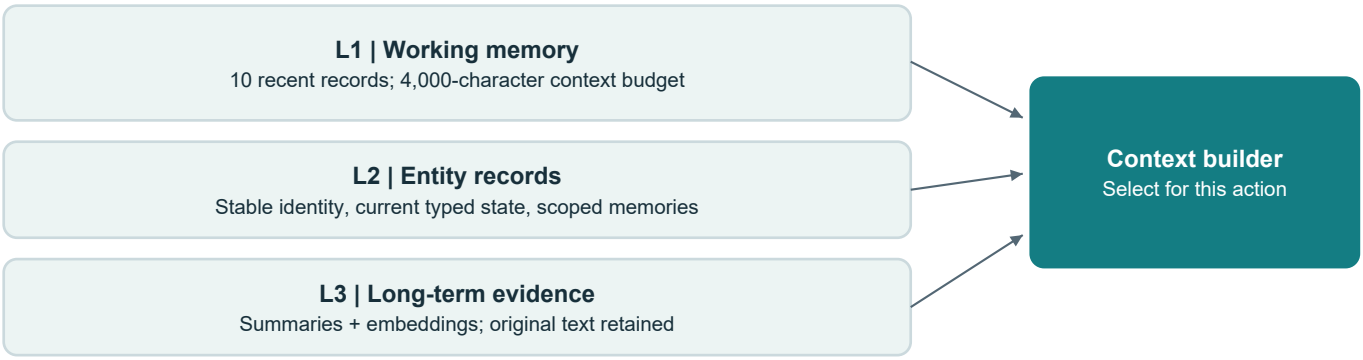


Figure 3. Memory is selected by purpose. Current structured facts remain authoritative. Historical summaries improve access to past events; original material remains available for provenance and recovery.

L1 - working memory. Keep up to 10 recent confirmed records per timeline. Select newer entries within a 4,000-character budget, then restore chronological order. Despite the name `L1_TOKEN_BUDGET`, the implementation counts Python string length, not tokenizer tokens. [12]

L2 - entity memory and state. Stable IDs link characters, registered attributes, relationships, status and scoped memories. Relevant entity records and current state enter the prompt. Ownership, location and health therefore need not depend solely on a regenerated prose summary, and rules can address values rather than ambiguous names. [12]

L3 - retrievable long-term memory. Evicted L1 records are summarized and optionally embedded; original text is stored separately. Model failure preserves raw content; embedding failure leaves a non-vector record. History folding retains the latest six lines. Epoch checks and compare-and-swap prevent stale compaction from overwriting newer history. [12]

These bounds limit injected history but do not establish a fixed total-token ceiling: world rules, entity context and other components also consume space. A future allocator should tokenize the complete request and measure evidence loss under each budget.

Return deterministic questions to Python

The trigger engine evaluates structured comparisons and Boolean trees. Scene equality, inventory checks, thresholds, entity-state comparisons, cooldowns and dependencies do not require free-form generation. AI conditions handle semantic uncertainty. AND, OR and NOT propagate an explicit unknown state; only resolved true authorizes a trigger. Unknown does not become true through negation. [12]

Question	Responsible component	Result
Is health above a registered threshold?	Python comparison	True / False / unknown
Does a valid local route exist?	Spatial validation	Allowed / blocked / review
Is an utterance dialogue or action?	Typed decision provider	Enumerated intent
What could happen next?	Apertus generation	Draft alternatives
May this proposal change the world?	Version checks + transaction	Commit once or reject

Typed model decisions use fixed question keys, candidate enums, numeric bounds and short evidence. Invalid or uncertain answers remain unresolved; fallback receives only unresolved questions. Kev-4B is the packaged local decision model, with Apertus fallback. Historical latency tests instead used Apertus 8B then 70B. In three revised routing samples, valid 8B source answers remained uncertain, adding 3.40-4.19 s of 70B fallback. Format validity is not decision quality. [4,9,12]

The state transition is **S(t+1) = Apply(S(t), validated committed effects)**. Drafts and streamed prose are outside that transition. The model handles language reasoning; functions handle exact, testable state changes.

6. Does the model have the evidence it needs?

Corpus, token count and protocol

The internal *Guang Yu Ye* campaign has **22 texts**: 18 chapters and four supplements, totaling **927,910 Unicode characters**. It produced 1,838 RAG chunks, 27 projected Wiki pages and 1,593 evidence blocks. A local recount matching all 22 archived normalized-content hashes measured **674,664 BGE-M3 tokens**, summed per document without special tokens or truncation. The longest text is 107,257 characters; the largest tokenized document is 78,640 tokens. The tokenizer revision matches `helpers/models.json`. This is source-corpus size, not Apertus input usage or reranker billing. [6,10]

The **48 fixed cases** comprise 22 basic questions, five deep-chapter questions, three cross-document questions and 18 synthetic boundary cases. Material questions contain **33 annotated facts across 30 questions**. Gold passages and offsets were fixed before retrieval; input hashes guard against changing materials. Hit@K counts questions with at least one gold match. Recall@K counts recovered annotated facts, so a question hit can remain incomplete. [6,7]

Hosted reranking results

K	Initial Hit@K	Reranked Hit@K	Initial Recall@K	Reranked Recall@K
1	21/30 (70.00%)	29/30 (96.67%)	21/33 (63.64%)	29/33 (87.88%)
3	25/30 (83.33%)	29/30 (96.67%)	26/33 (78.79%)	30/33 (90.91%)
6	25/30 (83.33%)	30/30 (100.00%)	26/33 (78.79%)	31/33 (93.94%)

Table 1. Actual path, including fallback. The model was hosted `mixedbread-ai/mxbai-rerank-v3.1-listwise`. The intervention includes an expanded BM25 shortlist and reranking, so it is not a model-only ablation. Live embeddings and narrative generation were disabled in this benchmark. [7]

Across 48 cases, 44 service calls succeeded. Two payloads differed from their prepared hashes and fell back before transmission; two cases had no candidates. The genuinely reranked material subset achieved Hit@6 **28/28** and Recall@6 **29/31**. Dynamic-prompt assembly coverage increased from **26/33 to 31/33**. Neither path returned the corresponding marked forbidden evidence across the 48 cases. [7]

Retrieval timing, 48 sequential cases	Median	p95, nearest-rank	Maximum
Initial retrieval	0.277 s	0.540 s	0.568 s
Actual path with reranking	2.905 s	6.972 s	8.246 s
Service calls only, n=44	1.920 s	6.387 s	7.539 s

The quality gain costs **2.628 s in median retrieval latency**, or 10.48x the initial median. Requests carried 3,441 fragments and 2,035,896 characters, including repeated evidence across queries. The service reported **527,289 input/total tokens and zero completion tokens**. This cumulative usage differs from the 674,664-token unique corpus. Timings exclude narrative generation. [7,10]

What the remaining misses reveal

The 80-fragment pool covered **32/33** facts. G28's missing cross-source evidence never entered the pool; G29's other fact remained below the final six. No previously recovered annotated facts were lost. Thus 30/30 Hit@6 does not mean 30 complete answers. These results demonstrate better evidence access, not answer correctness, hallucination reduction or local Base v2 quality. [7]

7. Open models, deployment and reproduction

Model roles and measurement boundaries

Component	Submitted configuration	Role
Narrative core	swiss-ai/Apertus-v1.5-70B	Summaries, selected prose, Wiki and fallback; used in latency traces
Embedding	BAAI/bge-m3	Local 1,024-dimensional dense retrieval; MIT
Reranker	mixedbread-ai/mxbai-rerank-base-v2	Local ordering; Apache-2.0; different from Table 1
Decisions	jaredpalmer/kev-4b on Qwen/Qwen3.5-4B-Base	Local typed decisions, Apertus fallback; Apache-2.0
Optional local generator	swiss-ai/Apertus-v1.5-8B	Separate GPU profile, not the measured 70B endpoint

Apertus is the generation foundation. Helpers have narrower retrieval and decision roles. Kev is a separate open model, not hosted Jev weights. Auxiliary revisions are pinned in `helpers/models.json`; Kev source is fixed to `1f04cfa28e6a708485b28a9585010b9e436b776e`. Model licences remain upstream. [1-4,12]

From the repository root, copy `track_2b/.env.docker.example` to `track_2b/.env.docker`, set `LLM_NAME`, `LLM_BASE_URL` and `LLM_API_KEY`, then run `make run`. Open `http://localhost:8000/index.html`. Compose starts the workbench and three local helpers, publishing only the workbench's loopback port. A volume persists SQLite and saves; `make stop` preserves it. [12]

`make smoke` checks the isolated application chain with mocked inference; `make smoke-live` exercises real services. `make prepare-model` and `make run-local` provide a separate local Apertus profile after model access is arranged. Initial builds require network access. Fully disconnected operation and concurrent real-model execution on target hardware remain unvalidated. Auxiliary weights total about **12.72 GB**; 32 GB Docker RAM and 40 GB disk are planning allowances, not measured minima. [12]

Data and reproduction

The public package contains an original CC0 lighthouse demo, separate from the internal evaluation campaign. Production saves, private source text and credentials are excluded. The aggregate evidence ledger records benchmark hashes, per-document token counts and tokenizer provenance without distributing the campaign. Historical scripts and private inputs remain developer artifacts; public demo smoke tests do not reproduce the 48-case benchmark. [6,7,10,12]

Retained records report 262 passing Python regressions, five frontend API tests, JavaScript syntax checks and workflow checks at 1365px and 390px. Coverage includes failed generation, interrupted streams, repeated operations, team isolation, rollback and commit behavior. These are historical records, not a fresh full regression run for this documentation update. Packaged offline smoke and Compose validation are also recorded; real helper quality requires separate validation. [9,12]

8. Limitations and next steps

Evidence is limited to a fixed Chinese benchmark and a few sequential live traces. There is no controlled full-context ablation, concurrent load study, user waiting-time study or end-to-end factual-answer evaluation. Hosted retrieval results cannot establish local Base v2 quality; English Kev evaluations do not establish Chinese roleplay reliability. Fresh model measurements are still needed on the final deployed revision.

Next steps are to run paired tests separating options, first text, completion, tokens and validity; evaluate local Base v2 and Kev on held-out Chinese cases; improve candidate recall and cross-source coverage; and test identity enforcement and concurrent deployment on target hardware. The inspected submission had no recorded demo video. The contribution is a traceable division of responsibility between generation, evidence and deterministic world management.

References and evidence provenance

External documentation was checked on 16 October 2026. Local labels refer to retained development artifacts. Only aggregate, non-textual evidence is distributed. The reviewed implementation is based on source commit **5587da7**; this report update is subsequent to it.

Models and submission

[1] Swiss AI Initiative. **Apertus v1.5 70B model card.** Model identity, licence and context capacity. https://huggingface.co/swiss-ai/Apertus-v1.5-70B

[2] BAAI. **BGE-M3 model card and pinned tokenizer.** Dense dimensions and tokenizer used in the recount. https://huggingface.co/BAAI/bge-m3 . Revision: `5617a9f61b028005a4858fdac845db406aefb181`.

[3] Mixedbread. **MXBAI Rerank Base v2 model card.** Local reranker identity and licence. https://huggingface.co/mixedbread-ai/mxbai-rerank-base-v2

[4] Jared Palmer. **Kev-4B model card.** Typed decisions and base-model dependency. https://huggingface.co/jaredpalmer/kev-4b

[5] HackApertus. **Track 2b submission requirements.** Six-page body limit and deliverables. https://github.com/HackApertus/project-template/blob/main/track_2b/README.md

Retained experimental evidence

[6] **Large-campaign retrieval evaluation**, 15 October. `Tech_report/GuangYuYe_Retrieval_Evaluation_2026-10-15.md` and `Tech_report/encyclopedia_guangyuye_results/offline.json`. Corpus, annotations and baseline. Case SHA-256: `8e68d75abaa74299e111980451624289c7509ed3c6d25564ec5dae8f2aa2316f`.

[7] **Reranker integration and live comparison**, 15 October. `Tech_report/Reranker_Integration_2026-10-15.md`, `Tech_report/Mixedbread_Rerank_Connection_2026-10-15.md` and `Tech_report/mixedbread_rerank_results/{live,analysis}.json`. Actual model, fallback, ranking, coverage, timing and usage.

[8] **Original live inference profile**, 16 October. `Tech_report/Inference_Latency_Live_2026-10-16.md` and `Tech_report/inference_benchmark/live_analysis.json`. Original generation lengths, caching and workflow timing.

[9] **Staged inference fixes and live traces**, 16 October. `Tech_report/Inference_Fixes_2026-10-16.md` and `Tech_report/inference_fix_results/{live,live_constrained}/results.json`. Two successes, one rejection, timing boundaries and historical regression results.

[10] **Report evidence ledger and local token recount**, 16 October. `docs/report_metrics.json`. All 22 source strings were recovered read-only from a retained test database, restored at their boundaries and checked against archived normalized-content hashes. BGE-M3 tokenizer; tokenizers 0.23.3; no truncation, padding or special tokens; per-document totals summed. Tokenizer SHA-256: `21106b6d7dab2952c1d496fb21d5dc9db75c28ed361a05f5020bbba27810dd08`. No campaign text was sent to a service during the recount.

[11] **Team technical-report brief**, 16 October. The report request and retained development note supply the 60-to-18-second, 40-to-5-second and over-80% perceived-wait observations. No paired sample protocol accompanies them.

[12] **Apertus Mundus implementation and deployment.** `src/agent.py`, `inference_contracts.py`, `engine.py`, `selected_branches.py`, `knowledge.py`, `wiki_compiler.py`, `memory_service.py`, `trigger.py`, `providers.py`; `helpers/models.json`; Compose definitions; `docs/DOCKER.md`; `src/scripts/smoke_deployment.py`. Implementation paths are relative to `track_2b/src/` when only a Python filename is given. Other paths are relative to `track_2b/`, except the explicit developer-only `Tech_report/` paths.